

Instruções de Entrega do Desafio

GIT, GIT FLOW, CI PIPELINE & TRABALHO COLABORATIVO NO GOOGLE DOCS

Este documento detalha o formato exato exigido para a entrega do desafio do Dashboard de Observabilidade. O objetivo principal deste projeto é consolidar na prática os conceitos de **Git Flow**, trabalho colaborativo síncrono/assíncrono e **Integração Contínua (CI)**.

1. Ferramenta Oficial de Relatório: Google Docs

Por que utilizar o Google Docs?

Como o desafio é realizado em **dupla**, vocês devem criar e estruturar o relatório utilizando o **Google Docs**. Isso permite que ambos trabalhem simultaneamente no mesmo arquivo, adicionando suas respectivas evidências de forma organizada e em tempo real, evitando conflitos de arquivos locais.

*Formato de Entrega: Ao finalizar, exportem o documento do Google Docs como **PDF** e façam o envio no portal acadêmico.*

2. Estrutura Exigida do Relatório

O documento entregue não pode conter apenas um resumo geral. Ele deve documentar **todas as 20 funcionalidades (passos) do desafio**, divididas igualmente entre a dupla (10 passos para cada).

Para cada um dos 20 passos, vocês devem fornecer obrigatoriamente:

1. **Explicação Curta do Processo:** O que está sendo adicionado ou configurado nesta etapa e qual o seu papel no painel de observabilidade.
2. **Prints do Terminal:** Captura contendo os comandos Git de criação da branch, adição dos arquivos, commit e push remoto.
3. **Prints do Pull Request (PR) e Aprovação:** Evidência na plataforma (GitHub/GitLab) mostrando que a branch correspondente foi submetida, que a pipeline de CI rodou e que seu parceiro revisou e aprovou (Approved) o código antes de mesclar na `develop`.
4. **Prints da Vercel:** O status do Deploy correspondente à atualização e como o painel está evoluindo visualmente a cada etapa de deploy contínuo.

3. Comandos de Git e Fluxo de Trabalho

Para cada tarefa nova desenvolvida pela dupla, os seguintes comandos devem ser executados e documentados:

Criando e Entrando na Branch Separada

```
git checkout -b feature/nome-da-funcionalidade
```

O que faz: Cria uma nova linha de desenvolvimento ramificada da branch principal e muda seu terminal para trabalhar nela.

Por que usar: Garante isolamento completo. Nenhuma modificação feita aqui afetará o código estável até que ele passe por revisão.

Adicionando Alterações ao "Palco"

```
git add .
```

O que faz: Prepara todas as modificações do diretório de trabalho atual para serem incluídas no próximo ponto de salvamento.

Por que usar: Permite ao Git saber exatamente quais arquivos modificados ou criados devem ser empacotados.

Criando um Ponto de Salvamento (Commit)

```
git commit -m "feat: adiciona kpi de uptime"
```

O que faz: Grava as alterações preparadas no histórico permanente do seu repositório local com uma mensagem curta explicativa.

Por que usar: Cria um registro histórico preciso. Em caso de erros, é possível reverter o projeto exatamente para este ponto estável.

Enviando a Branch para a Nuvem

```
git push origin feature/nome-da-funcionalidade
```

O que faz: Transfere a sua branch de desenvolvimento local para o servidor remoto do GitHub ou GitLab.

Por que usar: Torna suas alterações acessíveis ao seu colega de dupla e aciona o ambiente de testes de Integração Contínua (CI).

4. O Papel e Funcionamento da Pipeline de CI

A pipeline de CI configurada no Passo 3 é a ferramenta de proteção do projeto. Vocês devem documentar e explicar cada linha do arquivo de configuração de forma simples:

```
name: CI Validate on: pull_request: branches: - develop - main jobs: lint-and-test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - name: Validate Code run: | echo "Simulando teste automatizado do código..."
```

O que cada etapa faz e por quê:

- **name: CI Validate**

Por quê: Nomeia o fluxo de trabalho. Ajuda a dupla a identificar de forma rápida e visual na interface qual validação está rodando no momento.

- **on: pull_request: branches: [develop, main]**

Por quê: Define os gatilhos. Indica ao GitHub para disparar estes testes de forma totalmente automática sempre que um novo Pull Request for aberto para as ramificações estáveis de desenvolvimento ou produção.

- **runs-on: ubuntu-latest**

Por quê: Aloca uma máquina virtual limpa e padronizada em nuvem (com Linux Ubuntu) para executar os testes, garantindo que o código se comporte de maneira idêntica independentemente da máquina local da dupla.

- **uses: actions/checkout@v3**

Por quê: Clona o código-fonte atualizado que está no Pull Request para dentro do servidor de testes. Sem este passo, a máquina virtual estaria vazia e não conseguiria acessar os arquivos do projeto.

- **run: [comandos de validação]**

Por quê: É a execução real do teste. Roda linters, análises sintáticas ou verificações que garantem que não existam tags abertas incorretamente ou erros estruturais no HTML e CSS antes de permitir o merge.